Indian Institute of Technology Delhi
Department of Chemical Engineering

ThermoCubic: Real Gas PVT, Enthalpy, Entropy and Fugacity using Cubic Equations of State

Author: Jiya Ray
Entry No.: 2025CH71744
Dep.: Chemical Engg.
E-mail: ch7251744@iitd.ac.in

Author: Vaatsalya Paliwal
Entry No.: 2025CH71989
Dep.: Chemical Engg.
E-mail: ch7251989@iitd.ac.in

November 2026

Abstract

Our project is a Python program called ThermoCubic. It **predicts the PVT behaviour of a pure gas from four cubic equations of state and also gives the enthalpy H, entropy S and pure-component fugacity**. The program runs as a small website in the browser, with a calculator, a side-by-side comparison of the four equations, graphs and a saved history.

1 Objective

Our aim was to build a real-gas program based on a cubic equation of state (EOS) that gives the PVT behaviour, and then add **(i) enthalpy H and entropy S** and **(ii) the fugacity of a pure gas**. We built it in Python with a simple browser interface: you pick a gas, an EOS, a temperature and a pressure, and the program gives Z, molar volume, fugacity coefficient, fugacity, H and S, with a short note on what each number means.

2 Equations used

We took all the formulas from Smith, Van Ness, Abbott and Swihart, *Introduction to Chemical Engineering Thermodynamics*, 8th ed. The book writes the four EOS in one generic form (Sec. 3.6, Table 3.1):

$$P \;=\; \frac{RT}{V-b} \;-\; \frac{a(T)}{(V+\varepsilon b)(V+\sigma b)} \tag{1}$$

With $\beta = \Omega P_r/T_r$ and $q = \Psi\,\alpha(T_r)/(\Omega T_r)$, the compressibility factor satisfies

$$Z \;=\; 1+\beta \;-\; \frac{q\,\beta\,(Z-\beta)}{(Z+\varepsilon\beta)(Z+\sigma\beta)} \tag{2}$$

This is a cubic in Z, and we solve it directly for its real roots. When there are three real roots, the largest is the vapour and the smallest is the liquid. The program picks the stable phase as the root with the lowest $\ln\phi$. The parameters ε, σ, Ω, Ψ and $\alpha(T_r)$ come from Table 3.1.

Fugacity (Eq. 13.74 and 10.33):

$$\ln\phi \;=\; Z-1-\ln(Z-\beta)-qI, \qquad f=\phi P, \qquad I=\frac{1}{\sigma-\varepsilon}\ln\!\left(\frac{Z+\sigma\beta}{Z+\varepsilon\beta}\right), \tag{3}$$

and $I=\beta/Z$ for van der Waals.

Residual properties. The book gets these from the Chapter 6 integrals of the P-V-T relation. Putting the generic cubic into those integrals gives the closed forms below, which were also checked numerically against the fugacity formula (Section 6):

$$\frac{H^R}{RT} \;=\; Z-1+\left[\frac{d\ln\alpha}{d\ln T_r}-1\right]qI, \qquad \frac{S^R}{R} \;=\; \ln(Z-\beta)+\left[\frac{d\ln\alpha}{d\ln T_r}\right]qI \tag{4}$$

For SRK and PR,

$$\frac{d\ln\alpha}{d\ln T_r} \;=\; -\,\frac{\kappa\sqrt{T_r}}{1+\kappa\left(1-\sqrt{T_r}\right)}\,; \tag{5}$$

for RK it is $-1/2$ and for van der Waals it is 0.

Total H and S. The ideal-gas part uses

$$\frac{C_p^{ig}}{R} \;=\; A+BT+CT^2+DT^{-2} \qquad \text{(Table C.1)}, \tag{6}$$

integrated from the reference state $T_0 = 298.15$ K, $P_0 = 1$ bar, where $H = S = 0$:

$$H = H^{ig} + H^R, \qquad S = S^{ig} - R\ln\frac{P}{P_0} + S^R, \qquad H^{ig} = R\int_{T_0}^{T}\frac{C_p^{ig}}{R}\,dT, \qquad S^{ig} = R\int_{T_0}^{T}\frac{C_p^{ig}}{R}\,\frac{dT}{T}. \tag{7}$$

We took the critical constants and acentric factors from Table B.1, and built in eleven gases (methane, ethane, propane, n-butane, ethylene, CO_2, N_2, O_2, H_2, NH_3, water). H and S are not absolute values: they are measured from a reference state, an ideal gas at 298.15 K and 1 bar where both are set to zero, so only differences between states are physically meaningful.

3 A worked example

As a worked example, take n-butane at $T = 350$ K and $P = 9.4573$ bar with the Redlich–Kwong equation (Example 3.9 of the book). Table B.1 gives $T_c = 425.1$ K, $P_c = 37.96$ bar, $\omega = 0.200$. Our program follows the same steps as below.

Step 1: reduced properties.

$$T_r = \frac{T}{T_c} = \frac{350}{425.1} = 0.8233, \qquad P_r = \frac{P}{P_c} = \frac{9.4573}{37.96} = 0.2491$$

Step 2: α, β and q. For RK, $\alpha(T_r) = T_r^{-1/2}$ and $\Omega = 0.08664$, $\Psi = 0.42748$ (Table 3.1):

$$\alpha = T_r^{-1/2} = 1.1021, \qquad \beta = \frac{\Omega P_r}{T_r} = \frac{0.08664 \times 0.2491}{0.8233} = 0.02622, \qquad q = \frac{\Psi\alpha}{\Omega T_r} = \frac{0.42748 \times 1.1021}{0.08664 \times 0.8233} = 6.604$$

Step 3: solve the cubic. Putting these into Eq. (2) gives

$$Z^3 - Z^2 + 0.14624\,Z - 0.0045394 = 0$$

with three real roots: $Z = 0.83049$, 0.12620 and 0.04331.

Step 4: pick the stable phase. With $I = \ln\!\left(\frac{Z+\beta}{Z}\right)$ for RK, Eq. (3) gives $\ln\phi$ for the three roots as -0.1570, 0.1824 and -0.0137. The largest root $Z = 0.83049$ has the lowest $\ln\phi$, so the stable phase is the vapour. The middle root is unstable and is never reported. Then

$$V = \frac{ZRT}{P} = 2555 \text{ cm}^3/\text{mol}, \qquad \phi = 0.8547, \qquad f = \phi P = 8.084 \text{ bar}.$$

Step 5: H and S. For RK, $d\ln\alpha/d\ln T_r = -1/2$, so Eq. (4) gives $H^R = -1389.2$ J/mol and $S^R = -2.664$ J/(mol K). The C_p constants of n-butane from Table C.1 are $A = 1.935$, $B = 36.915 \times 10^{-3}$, $C = -11.402 \times 10^{-6}$, $D = 0$. Integrating from the reference state gives $H^{ig} = 5473.9$ J/mol and $S^{ig} - R\ln(P/P_0) = -1.780$ J/(mol K). Adding the two parts:

$$H = 5473.9 - 1389.2 = 4084.7 \text{ J/mol}, \qquad S = -1.780 - 2.664 = -4.444 \text{ J/(mol K)}.$$

These are exactly the numbers the program prints for this case.

4 Method and assumptions

This is how our program does one calculation: (1) convert the input to K and bar; (2) find T_r, P_r, α, β and q; (3) solve the cubic and keep real roots with $Z > \beta$; (4) compute $\ln\phi$, H^R and S^R for every root; (5) report the root with the lowest $\ln\phi$ as the stable phase; (6) add the ideal-gas parts to get H and S. When only one real root exists the result is labelled "single real root" (or "supercritical" if T_r is 1 or more). The program does not claim a phase for a single root.

We assume a pure component, a single phase at the reported root, and constant critical data. Inputs must be positive numbers (T up to 5000 K, P up to 2000 bar); anything else gives an error message. The program shows a warning when: T outside 298.15 K to $T_{\max}$ of the C_p fit (Table C.1); near the critical point ($|T_r - 1| < 0.05$ and $|P_r - 1| < 0.3$); P_r above 3; three real roots; or liquid and vapour fugacities within 0.2 percent, which only means the pressure is close to the saturation pressure for that equation (not a confirmed two-phase state). The program does not search for the saturation pressure itself.

5 How we built the program

The program has three parts: `eos.py` holds the equations (root finding, ϕ, H, S), `app.py` is a small web server that uses only the Python standard library and stores the history in a SQLite file, and the `web` folder holds the pages (HTML, CSS and JavaScript). When the user presses Calculate, the page sends the gas, EOS, T and P to the server through a small JSON link (`/api/calc`), which calls `eos.py` and returns the numbers as JSON; the compare table and the graphs ask for arrays of points in the same way. All graphs are drawn as SVG in the browser by a small chart function in the page's JavaScript, so no plotting or web framework library is used. History rows are stored in the SQLite file and can be downloaded as a CSV file.

Development steps

1. The assignment requirements and textbook were used to define the calculations: PVT behaviour, enthalpy, entropy and pure-component fugacity.
2. The Python calculation engine implements the four cubic equations of state.
3. A browser interface was added for inputs, results, model comparisons, graphs and saved history.
4. The program was checked against worked examples in the textbook and thermodynamic consistency tests.
5. For the online version, the Python calculations were translated into JavaScript with AI assistance. This version runs directly in the browser without starting a Python server and is hosted on GitHub Pages at **https://jiya-ray.github.io/ThermoCubic/**
6. Automated comparisons covered 344 test cases, with matching results between the Python and browser versions, including phase labels and warning messages. The online version saves history in the browser.

To run the software locally

Open the `RealGasStudio` folder in VS Code and run `python app.py` in its terminal. Python 3.6 or later is required. The app opens in the browser at `127.0.0.1:8000`; no extra packages are needed.

The main calculation code is shown below in four parts. Lines left out are marked with `...`; the full code is in the submitted ZIP (`eos.py`, `app.py`, `web/`). The four equations differ only in five constants, so one table covers all of them.

1. Equation parameters and gas data:

```
EOS = {
    "vdW": dict(name="van der Waals", eps=0.0, sig=0.0, omega=1/8, psi=27/64),
    "RK": dict(name="Redlich-Kwong", eps=0.0, sig=1.0, omega=0.08664, psi=0.42748),
    "SRK": dict(name="Soave-Redlich-Kwong", eps=0.0, sig=1.0, omega=0.08664, psi=0.42748),
    "PR": dict(name="Peng-Robinson", eps=1-math.sqrt(2), sig=1+math.sqrt(2), omega=0.07780, psi=0.45724),
}

GASES = {
    "Methane": dict(Tc=190.6, Pc=45.99, w=0.012, cp=(1.702, 9.081e-3, -2.164e-6, 0.0)),
    ... (11 gases, each with Tc, Pc, acentric factor and the four Cp constants)
}
```

2. The α function and its temperature derivative $d\ln\alpha/d\ln T_r$ (both are needed for H^R and S^R):

```
def alpha_terms(model, Tr, w):
    if model == "vdW":
        return 1.0, 0.0
    if model == "RK":
        return Tr ** -0.5, -0.5
    if model == "SRK":
        k = 0.480 + 1.574 * w - 0.176 * w * w
    else:
        k = 0.37464 + 1.54226 * w - 0.26992 * w * w
    s = 1 + k * (1 - math.sqrt(Tr))
    return s * s, -k * math.sqrt(Tr) / s
```

3. The solver: it builds the cubic coefficients, finds the real roots in closed form (the trigonometric formula when there are three), polishes each root with Newton steps, and works out $\ln\phi$, H^R and S^R for every root. The root with the lowest $\ln\phi$ is reported as the stable phase:

```
def solve(model, gas, T, P):
    ...
    c2 = s * B - 1 - B
    c1 = pr * B * B - (1 + B) * s * B + A
    c0 = -(1 + B) * pr * B * B - A * B
    roots = [z for z in cubic_roots(c2, c1, c0) if z > B]
    ...
    for z in roots:
        if m["sig"] == m["eps"]:
            I = B / z
        else:
            I = math.log((z + m["sig"] * B) / (z + m["eps"] * B)) / (m["sig"] - m["eps"])
        lnphi = z - 1 - math.log(z - B) - q * I
        hr = (z - 1 + (dla - 1) * q * I) * R * T
        sr = (math.log(z - B) + dla * q * I) * R
```

4. The ideal-gas part: the C_p polynomial integrated from the reference state, in the same way:

```
def ideal_part(gas, T, P):
    a, b, c, d = GASES[gas]["cp"]
    h = a * (T - T0) + b / 2 * (T * T - T0 * T0) + c / 3 * (T ** 3 - T0 ** 3) - d * (1 / T - 1 / T0)
    s = a * math.log(T / T0) + b * (T - T0) + c / 2 * (T * T - T0 * T0) - d / 2 * (1 / T ** 2 - 1 / T0 ** 2)
    return h * R, s * R - R * math.log(P / P0)
```

The tables in Section 7 are produced by this program.

6 Checks

(a) *Book example.* We compared the program with Example 3.9 of the book, which solves n-butane at 350 K and 9.4573 bar (saturated vapour and liquid) with the RK equation. The program gives $Z = 0.8305$ ($V = 2555$ cm^3/mol) and $Z = 0.04331$, **matching the book's 0.8305 (2555 cm^3/mol) and 0.04331**.

(b) *Ideal-gas limit.* At a very low pressure ($P = 10^{-4}$ bar), Z comes out as 1 within 10^{-4} for every EOS and gas tested.

(c) *Thermodynamic consistency.* For all four EOS, four gases and three (T, P) points each we checked that

$$H^R = -RT^2 \left(\frac{\partial \ln \phi}{\partial T} \right)_P \text{ (by finite difference)} \qquad \text{and} \qquad G^R = H^R - TS^R = RT \ln \phi. \tag{8}$$

All checks passed, so H, S and ϕ agree with each other.

7 Sample results

The first table gives n-butane at 350 K and 9.4573 bar for all four equations, and the second gives four more cases with Peng–Robinson. The stable phase is picked from the lowest $\ln \phi$:

EOS	Z	V (cm^3/mol)	ϕ	f (bar)	H (J/mol)	S (J/mol K)
van der Waals	0.8667	2666.9	0.8829	8.350	4565.6	−3.340
Redlich–Kwong	0.8305	2555.3	0.8547	8.084	4084.7	−4.444
Soave–Redlich–Kwong	0.8191	2520.3	0.8469	8.009	3893.7	−4.913
Peng–Robinson	0.8081	2486.4	0.8374	7.920	3870.4	−4.887

Case (PR)	Z	ϕ	f (bar)	H (J/mol)	S (J/mol K)
Methane, 200 K, 30 bar	0.7554	0.7951	23.852	−4305	−44.90
CO_2, 320 K, 60 bar	0.6950	0.7548	45.289	−1973	−37.77
Propane, 400 K, 20 bar	0.8503	0.8648	17.296	7038	−2.92
Nitrogen, 300 K, 100 bar	0.9889	0.9736	97.364	−577	−39.99

The four EOS give Z between 0.808 and 0.867 for the butane case. Peng–Robinson is the lowest. **The fugacity coefficients are all below 1, so attraction between molecules dominates** at these conditions and the gas has a lower fugacity than pressure.

8 Features of the interface

The opening page shows the software name and our names. On the main page we kept a calculator with short explanations under each result, a table comparing all four EOS at the same T and P, a P-V isotherm plot, five property graphs (Z, f and ϕ against pressure; H and S against temperature), a saved history with notes and a CSV download, and a Learn page with the equations.

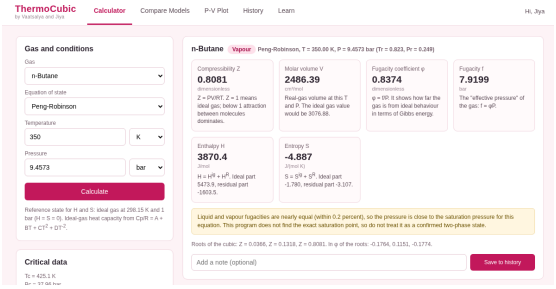


Figure 1: Calculator page, n-butane, Peng–Robinson, 350 K, 9.4573 bar.



Figure 2: Saved history with notes, CSV download and delete.



Figure 3: Graphs tab (n-butane, PR). Dashed lines are the ideal gas. A break in a line is a switch of the stable root between the vapour and liquid roots, so the points are not joined. The P-V plot shows the raw cubic curve including its unstable loop; the real equilibrium path at the saturation pressure is not drawn.

9 Limits and conclusion

Limits: the program covers pure gases only; mixtures are not handled. It does not calculate vapour pressure and uses constant critical data from the book. Near the critical point or at very high pressure the cubic EOS is less accurate, mainly for the liquid volume. The C_p^{ig} constants are valid from 298 K up to the $T_{\max}$ in Table C.1; outside that range the program warns. The model reproduces the book's RK example, and the H, S and fugacity values are consistent with each other. **This completes the extension to H, S and pure-component fugacity.**

Reference: J. M. Smith, H. C. Van Ness, M. M. Abbott, M. T. Swihart, *Introduction to Chemical Engineering Thermodynamics*, 8th ed., McGraw-Hill, 2018.